Dataset Used: kc_house_data.csv

The dataset used is **KC House Data**, which contains housing details and their corresponding prices.
It includes features like bedrooms, bathrooms, square footage, floors, condition, grade, and year built.
The target variable is **house price**, which is a continuous value.
This dataset is commonly used for **regression problems**, but in this project, classification algorithms are applied for learning and comparison purposes.

Goal: The main goal of this project is to apply and evaluate multiple machine learning algorithms on the KC housing dataset.

It aims to analyze how different models perform using housing features such as area, bedrooms, and condition.
The project also focuses on understanding the importance of data preprocessing and feature scaling.
Another objective is to compare algorithm performance and identify the most effective model.
Finally, it helps in understanding which type of approach (classification vs regression) is suitable for housing price prediction.

Modules Used:

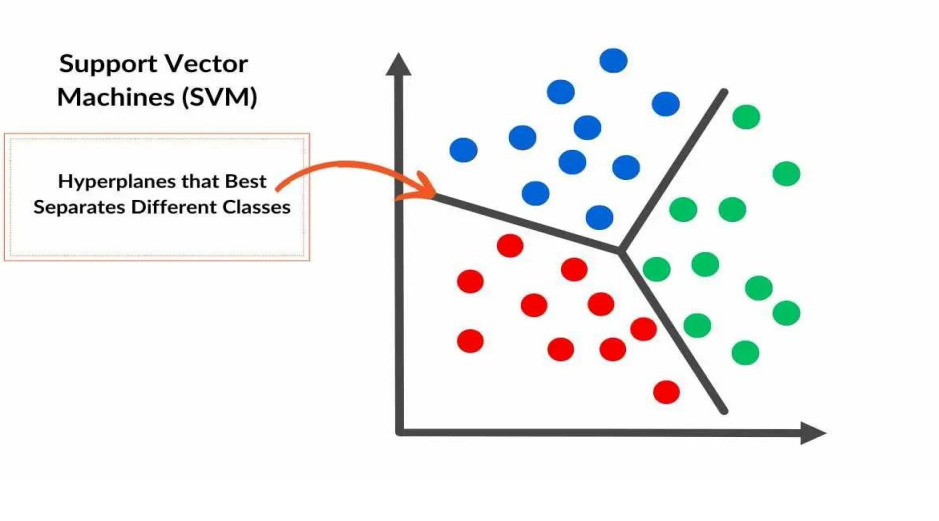
The **primary libraries/modules** required for this project are:

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (`np.diff`).
- **Scikit-Learn:** Scikit-learn is the main machine learning library used in this project. It provides tools for data splitting, preprocessing, model building, and evaluation. All classification algorithms used in this project come from this library.
- **train_test_split:** This module is used to divide the dataset into training and testing sets. It ensures that the model is evaluated on unseen data. In this project, 80% data is used for training and 20% for testing.
- **StandardScaler:** Standard Scaler is used for feature scaling. It standardizes features by converting them into a common scale. This improves model performance and prevents bias due to feature range differences.
- **Classification Algorithms (from Scikit-learn):**
    - SVC – Used for Support Vector Machine classification
    - Logistic Regression – Used for probability-based classification
    - Gaussian NB – Used for probabilistic classification
    - Decision Tree Classifier – Used for tree-based decisions
    - Random Forest Classifier – Used for ensemble learning
    - KNeighbors Classifier – Used for distance-based classification (KNN)
    - Gradient Boosting Classifier – Used for sequential ensemble learning
- **Accuracy_score:** This module is used to evaluate the model performance. It calculates the accuracy of predictions made by each algorithm. Helps in comparing different models.

Classification Algorithms Implemented in the Project

1. Support Vector Machine (SVM)

SVM is a supervised learning algorithm used for classification tasks. It works by finding the optimal hyperplane that separates different classes with the maximum margin. The data points closest to the boundary are called support vectors. It can also handle non-linear data using kernel functions. SVM is effective in high-dimensional spaces.. In this housing project, SVM attempts to classify houses into price-related categories using features such as bedrooms, square footage, and condition. After applying feature scaling, it analyzes patterns between housing features and output values. SVM performs well in high-dimensional data and can handle non-linear relationships using kernels. However, housing prices are continuous values, making classification less appropriate. It struggles when price ranges overlap significantly. Selecting the correct kernel and parameters is difficult. It is also computationally expensive for large datasets like KC housing data.



SVM Classifier

Limitations

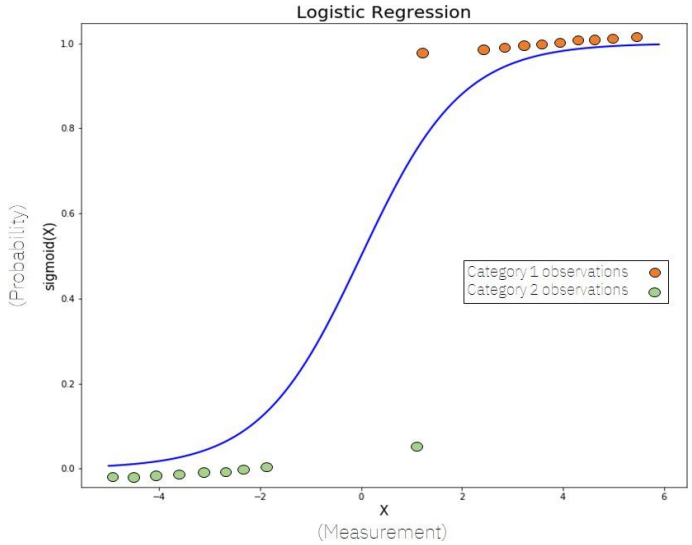
- Support Vector Machine is powerful, but it becomes computationally expensive when applied to large datasets, especially with non-linear kernels like RBF. Training time increases significantly as the number of samples grows.
- Selecting the appropriate kernel function and hyperparameters (C, gamma) is challenging and requires careful tuning, often through trial-and-error or cross-validation.
- SVM does not perform well when the dataset contains significant noise or overlapping classes, as it tries to strictly separate classes using margins.
- It is also less interpretable, meaning it is difficult to understand how the model makes decisions compared to models like Decision Trees.
- Additionally, SVM requires proper feature scaling, and performance may degrade if features are not normalized.

Conclusion:

SVM performs well when classes are clearly separable and feature scaling is applied. It is robust against overfitting in high-dimensional datasets. However, it can be computationally expensive for large datasets. In this program, it provides strong classification accuracy on the Iris dataset.

2. Logistic Regression

Logistic Regression is a classification algorithm that predicts probabilities using a sigmoid function. It models the relationship between input features and the probability of a class. The output is converted into class labels using a threshold. It works best when the data is linearly separable. It is used to classify house prices into categories based on features like area and number of rooms. It assumes a linear relationship between input features and the output. The model is simple, fast, and easy to interpret. It works well when the data is linearly separable. However, housing data often contains complex and non-linear relationships. Logistic Regression may oversimplify these patterns. It is sensitive to outliers such as luxury houses. It is not suitable for predicting exact price values.



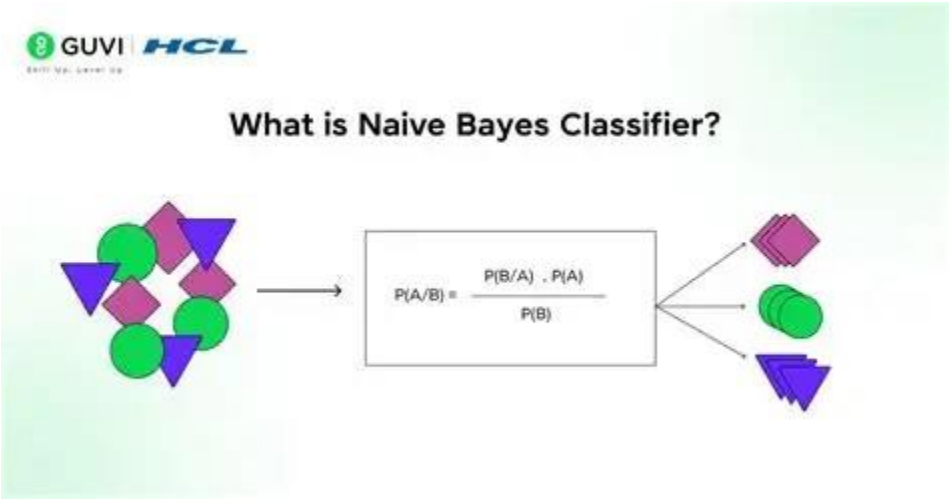
Limitations

- Logistic Regression assumes a linear relationship between independent variables and the log-odds of the dependent variable, which limits its performance on complex, non-linear datasets.
- It struggles when decision boundaries are not linear unless feature engineering or transformations are applied.
- The model is highly sensitive to outliers, which can significantly distort the decision boundary and affect predictions.
- Another limitation is multicollinearity, where highly correlated features can make coefficient estimates unstable and reduce interpretability.
- It may also underfit the data when relationships between features and target are complex, making it less accurate than advanced models like Random Forest.

Conclusion:
Logistic Regression is simple, fast, and easy to interpret. It performs well on linearly separable data but may struggle with complex patterns. It serves as a good baseline model for classification problems. In this program, it provides reliable and efficient results.

3. Naive Bayes (GaussianNB)

Naive Bayes is a probabilistic classifier based on Bayes' theorem. It assumes that all features are independent of each other. Gaussian Naive Bayes specifically assumes that features follow a normal distribution. It calculates the probability of each class and selects the most likely one.In this project, it predicts house price categories based on features like size, condition, and number of rooms. Gaussian NB assumes that the data follows a normal distribution. It is fast, efficient, and works well with high-dimensional data. However, housing features are often correlated, violating the independence assumption. This reduces model accuracy. The assumption of normal distribution may not hold true for housing prices. It may produce oversimplified predictions.



Naive Bayes (GaussianNB)

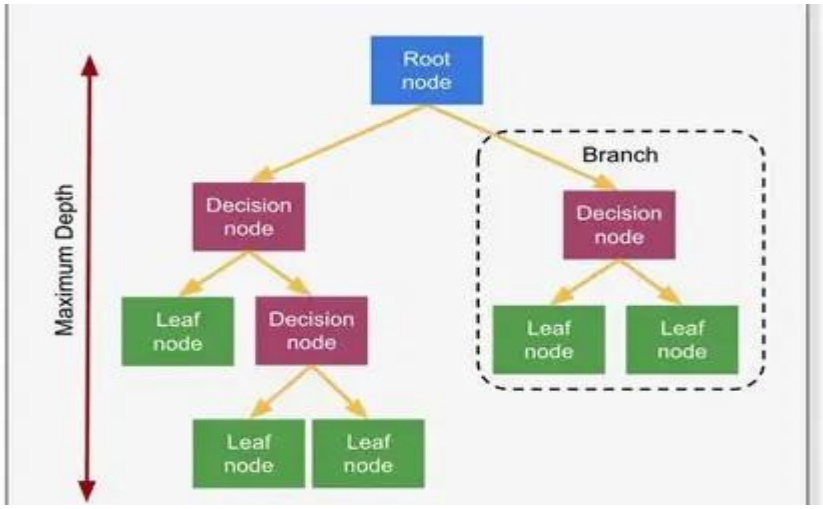
Limitations

- Naive Bayes relies on the strong assumption that all features are **independent**, which is rarely true in real-world datasets. This can reduce the model's effectiveness when features are correlated.
- GaussianNB assumes that the data follows a **normal (Gaussian) distribution**, which may not always hold true, especially in skewed datasets.
- The model may produce **lower accuracy** compared to more sophisticated algorithms when feature interactions are important.
- It can be sensitive to how data is distributed and may struggle with **imbalanced datasets**.
- Although it is fast and efficient, its simplicity can sometimes lead to **oversimplified predictions** that do not capture complex patterns.

Conclusion:
Naive Bayes is fast, efficient, and works well with small datasets. Despite its strong independence assumption, it often performs surprisingly well. It may be less accurate when features are highly correlated. In this program, it provides quick predictions with reasonable accuracy.

4. Decision Tree Classifier

Decision Tree is a non-linear model that splits data into branches based on feature values. It uses criteria like Gini index or entropy to decide the best splits. The structure resembles a flowchart where each node represents a decision. It can handle both numerical and categorical data. It creates rules such as "if square footage is high, then price category is high." In this project, it helps model non-linear relationships in housing data. It is easy to understand and visualize like a flowchart. It can handle both numerical and categorical data effectively. However, it is prone to overfitting, especially with complex housing features. Small changes in data can lead to different tree structures. It may not generalize well to new data. Deep trees can become complex and less interpretable.



Decision Tree Classifier

Limitations

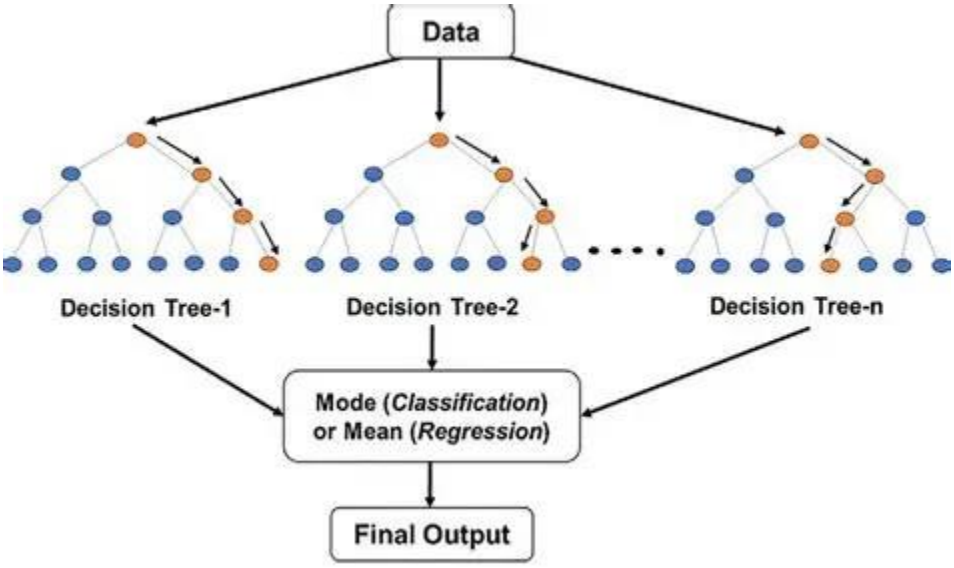
- Decision Trees are highly prone to overfitting, especially when the tree grows deep and captures noise instead of general patterns.
- They have high variance, meaning small changes in the dataset can result in completely different tree structures.
- Decision Trees may not generalize well to unseen data unless techniques like pruning or depth limitation are applied.
- They tend to create biased trees if the dataset is imbalanced, favoring dominant classes.
- Additionally, with many features, trees can become overly complex and harder to interpret, reducing their practical usability.

Conclusion:

Decision Trees are easy to understand and interpret. They can capture complex relationships in data but are prone to overfitting. Proper tuning is required to improve performance. In this program, it provides flexible classification but may vary in accuracy.

5. Random Forest Classifier

Random Forest is an ensemble learning algorithm that combines multiple decision trees. Each tree is trained on different subsets of housing data and features. The final prediction is made using majority voting. It improves accuracy and reduces overfitting compared to a single decision tree. In this project, it captures complex relationships between features like area, grade, and price. It is robust and performs well on large datasets. However, it requires more computational power. It is less interpretable than a single tree. It is still not ideal for continuous price prediction.



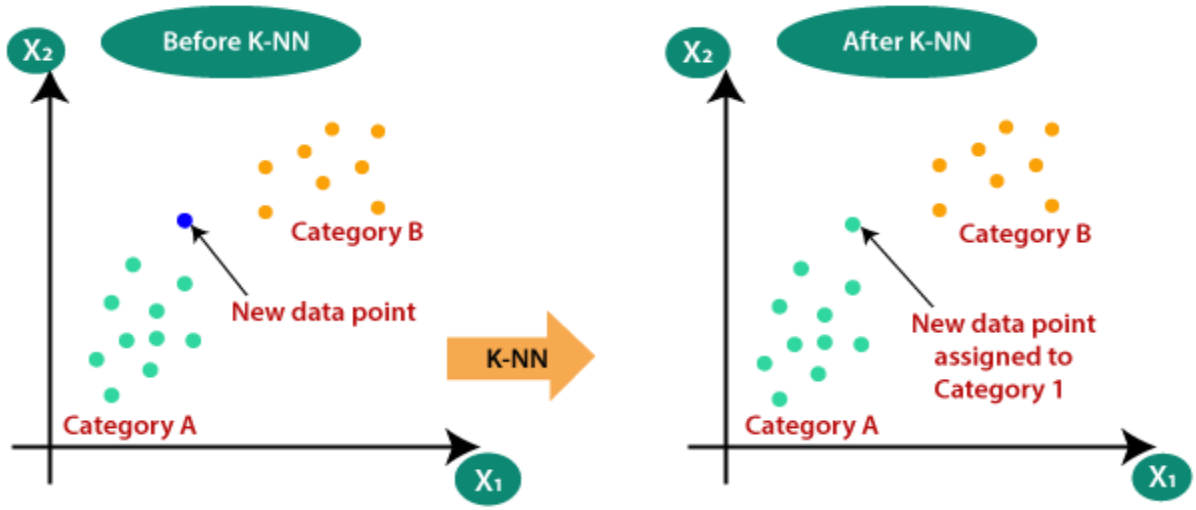
Random Forest Algorithm

Limitations

- Random Forest reduces overfitting but introduces increased computational complexity, as it builds multiple decision trees instead of one.
- Training can be slower, and the model requires more memory and processing power, especially with a large number of trees.
- It is less interpretable compared to a single Decision Tree, making it difficult to understand how predictions are made.
- Although robust, it may not perform optimally on very small datasets, where simpler models could be more effective.
- Proper tuning of hyperparameters like the number of trees, depth, and features is necessary, otherwise the model may become inefficient or unnecessarily complex

K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a distance-based classification algorithm. It predicts the output based on the majority class of the nearest data points. In this project, it compares a house with similar houses using features like area and number of rooms. It uses distance metrics such as Euclidean distance to measure similarity. It is simple and easy to implement. It works well when similar data points exist in the dataset. However, it is slow for large datasets like KC housing data. It is sensitive to feature scaling and outliers. Performance decreases with high-dimensional data.



Explaining KNN

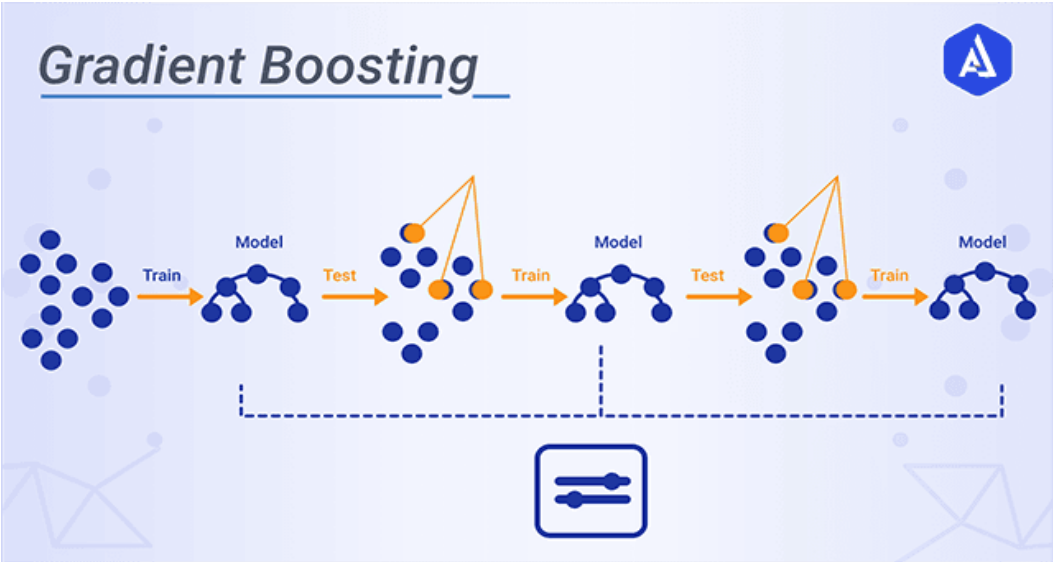
Limitations

KNN has several limitations that can affect its performance in real-world scenarios:

- **High Computational Cost**
  KNN is slow during prediction because it calculates the distance between the test point and all training data points. This becomes inefficient for large datasets.
- **Choice of K Value**
  Selecting the optimal K is difficult.
  - Small K → sensitive to noise (overfitting)
  - Large K → less accurate (underfitting)
- **Curse of Dimensionality**
  As the number of features increases, the distance between points becomes less meaningful, reducing model performance.
- **Sensitive to Feature Scaling**
  Features with larger values dominate distance calculations. Without normalization (like StandardScaler), results become biased.
- **Affected by Noise and Outliers**
  Noisy data or outliers can mislead the algorithm, especially when K is small.
- **Memory Intensive**
  Since it stores all training data, it requires more memory compared to other models.

Gradient Boosting Classifier

Gradient Boosting is an ensemble learning algorithm that builds models sequentially. Each new model corrects the errors made by previous models. It combines multiple weak learners, usually decision trees, to improve performance. In this project, it captures complex relationships between housing features and price categories. It provides high accuracy and strong predictive performance. It is widely used for structured data problems. However, training is slow due to its sequential nature. It requires careful tuning of parameters. It is sensitive to noise and outliers. Overfitting may occur if too many models are built.



Elaborating Gradient Boosting

Limitations

Despite its high performance, Gradient Boosting has several drawbacks:

- **High Computational Complexity**
  Training is slow because models are built sequentially. Each step depends on the previous one, making it time-consuming.
- **Requires Careful Hyperparameter Tuning**
  Parameters like:
  - Learning rate
  - Number of trees
  - Tree depth

  must be tuned properly. Poor tuning can reduce performance.
- **Risk of Overfitting**
  If too many trees are added or the learning rate is too high, the model may memorize training data instead of generalizing.
- **Sensitive to Noise and Outliers**
  Since each model tries to correct previous errors, noisy data can get amplified, affecting accuracy.
- **Less Interpretability**
  The model is complex and difficult to interpret compared to simpler models like Decision Trees.
- **Sequential Nature (No Parallelization)**
  Unlike Random Forest, Gradient Boosting cannot be easily parallelized, increasing training time.

Conclusion

In this project, multiple machine learning algorithms were applied to analyze housing data and compare their performance.

Each algorithm uses a different approach, such as linear models, probability-based methods, and ensemble techniques.

Ensemble models like Random Forest and Gradient Boosting performed better due to their ability to capture complex relationships.

Simple models like Logistic Regression and Naive Bayes showed limitations with non-linear housing data.

Since house price is a continuous variable, regression models would be more suitable for accurate prediction.

Final Conclusion

This project demonstrates a complete machine learning workflow, including data preprocessing, feature scaling, model training, and evaluation using the KC housing dataset.

Multiple classification algorithms were applied to understand and compare their performance on real-world data.

Among them, ensemble methods like Random Forest and Gradient Boosting showed better results due to their ability to handle complex feature relationships.

However, since house price is a continuous variable, classification models are not the most appropriate choice.

For more accurate and practical predictions, regression algorithms should be used in future work.

What we learned:

Through this project, we gained practical knowledge of applying machine learning algorithms to real-world housing data.
We learned how to **load, preprocess, and scale data** using tools like Pandas and Standard Scaler.
We understood how different algorithms work and how their performance varies based on data characteristics.
We also learned to **train models, make predictions, and evaluate results** using accuracy metrics.
Additionally, we realized the importance of choosing the right type of model, as regression is more suitable for continuous data like house prices